

Name: Manesh Nimsara Samararathne

Student Reference Number: 10638293

Module Code: PUSL2008

Module Name: Introduction to Internet of Things

Coursework Title: IoT Bases Smart Green House System

Deadline Date: 24 April 2019

Member of staff responsible for coursework:

Programme: BSc (Hons) Software Engineering

Please note that University Academic Regulations are available under Rules and Regulations on the University website www.plymouth.ac.uk/studenthandbook.

Group work: please list all names of all participants formally associated with this work and state whether the work was undertaken alone or as part of a team. Please note you may be required to identify individual responsibility for component parts.

M. N. Samararathne(Team-Lead)	10638293
W.P. Ashene M. Perera	10638292
Y.D.R. Prabha	10638231
G.P.T Dilshan	10638234
P.D.S.V. Jayarathne	10642823

We confirm that we have read and understood the Plymouth University regulations relating to Assessment Offences and that we are aware of the possible penalties for any breach of these regulations. We confirm that this is the independent work of the group.

Signed on behalf of the group: 

Individual assignment: ***I confirm that I have read and understood the Plymouth University regulations relating to Assessment Offences and that I am aware of the possible penalties for any breach of these regulations. I confirm that this is my own independent work.***

Signed :

Use of translation software: failure to declare that translation software or a similar writing aid has been used will be treated as an assessment offence.

I *have used/not used translation software.

If used, please state name of software.....

Overall mark _____ % Assessors Initials _____ Date _____

Declaration

When forwarding this submission, we claim that our work contains no examples of abuses such as plagiarism or paraphrase. All content has been properly processed in accordance with Plymouth guidelines. Below is the list of members of the group.

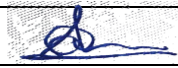
Team 08		
Member name	Plymouth ID	Signature
Manesh Nimsara Samarathne (Team-Lead)	10638293	
W.P. Ashene M. Perera	10638292	
Y.D.R. Prabha	10638231	
G.P.T Dilshan	10638234	
P.D.S.V. Jayarathne	10642823	

Table of Contents

Chapter 1 – Introduction	7
1.1 Purpose	7
1.2 Scope	8
1.3 Goal	9
1.4 Objectives	9
Chapter 2 – Literature Survey	10
2.1 Design and implementation of the smart greenhouse environment and plant condition controlling system	10
2.2 IOT based smart greenhouse environment and plant condition controlling system	10
2.3 IOT based greenhouse using Arduino	11
2.4 Technical review	11
Chapter 3 – Project Planning	13
3.1 Assumptions & constraints	13
3.2 Project management approach	13
3.3 Project Budget	14
Chapter 4 – Software Requirements Specification	15
4.1 Overall description	15
4.1.1 Product perspective	15
4.1.2 Product Features	15
4.1.3 Operating Environment	15
4.1.4 Class diagrams and Characteristics	16
4.1.5 Design and Implementation constraints	16
4.2 System Features	17
4.2.1 Description and Priority	17
4.3 Functional requirements	17
4.4 External Interface Requirements	18
4.4.1 User Interfaces	18
4.4.2 Hardware Interfaces (Sensors)	18

4.5	Non-functional Requirements	20
4.5.1	Performance Requirements.....	20
4.5.2	Safety Requirements	20
4.5.3	Security Requirements	21
Chapter 5 – System Design		22
5.1	System Requirement Definition	22
5.2	Functional requirements	23
5.3	Use case Diagram	23
5.4	Data Flow Diagram.....	24
5.5	Sub-system Development	26
5.5.1	Sensor testing	26
5.5.2	Wi-Fi Connectivity.....	26
5.5.3	Status Shown	26
5.5.4	Joining all together	26
5.6	Systems Integration.....	27
Chapter 6 – Implementation		28
6.1	Sensors module	28
6.2	Wi-Fi connectivity	33
6.3	Connecting to Thing speak Server	34
6.4	Communication between hardware & software	38
Chapter 7 – System Testing.....		44
7.1	Test Cases and Test Results	44
7.2	Sample of a Test Case for a Sensor.....	44
7.3	Software Quality Attributes	46

Chapter 8 – Conclusion & Future Scope	47
8.1 Future scope	47
Chapter 9 – Work Load Matrix	48
Chapter 10 – Peer Evaluation	49
References.....	50

Report Organization

Chapter 1

When preparing project most important part is Introduction of the project. Introduction be categorized by following things like motivation, definition of the problem relevant to the current system, advantages, goals and objectives, scope and application.

Chapter 2

Here we discussed problems overcome under every paper. Problem relevant to descriptions. At least we produced three papers. They are different from each other.

Chapter 3

In this chapter we discussed requirement analysis. Under this category we talk about requirements like supporting, software and hardware requirements with the probable study.

Chapter 4

In this chapter we consider about system design and architectural concept. We must be seen various diagrams that demonstrate the software. Diagrams include in software in various ways like use case diagram. DFD diagram, class diagram and component diagram.

Chapter 5

We need methodology, to explain the project in details and the help of a few diagram. Here we give priority divide the project into module for example: cab search

Chapter 6

This part we discussed details of the implementation of methodology refer to chap.5, It contains assumption and dependences implementation details.

Chapter 7

This part contained the result of the analytic discussion. Test cases and output of the project.

Chapter 8

We included whole project and future scope in this chapter.

Chapter 1 – Introduction

A Green house is a simple structure which is consist with walls and roof glazed by transparent materials such as glass, plastic, polycarbonate. Controlling the climate for the plants that grown inside the green house is a main purpose of the Green House. Also, it provides the protection of the plants from pests and harmful rays in the environment.

Greenhouse structures mainly divide in two categories like,

1. Attached greenhouse
2. Free standing greenhouse

The two structures differ each other from design, flexibility, location, expandable, size etc.

Glass is the most famous **glazing** method that use for greenhouses also plastic and polycarbonate are also used to glaze them. The main purpose of using transparent materials to the glazing is to get the sunlight and its warmth in to the greenhouse.

Gardening inside a greenhouse is a very different task than outside environment. To grow a healthy plant, it needs a proper water, air, temperature, sunlight supply and a proper media. In outside gardening it is very easy to find and not need to control those conditions mostly but outside gardening cannot done all over the year due to the climate changes that happens in most countries except tropical countries.

When gardening inside a greenhouse the farmers must consider about all the conditions that need to grow healthy plants, for that they need to set up equipment to check the condition levels. The farmers must always stay aware of those condition levels also.

1.1 Purpose

Today every industrial section of the world is facing the main problem with the concept of the Smart customer. To keep connection and fulfil these smart customers' requirements agricultural section must equipped with new technology to improve quality and production.

Owners of the greenhouses must track the condition levels and they must always stay aware about those conditions and make decisions and make actions according to them. Doing those functions manually will be very hard when managing with the other works of the greenhouse owner.

By this system the greenhouse owners only go to the farm only for setting up this system equipment. They can control the greenhouse environment and plant conditions by clicking few buttons on a GUI interface.

1.2 Scope

This internet of this project is about an internet based automated environmental and plant controlling system.

In this internet-based greenhouse environmental and plant controlling system it controls the following major environmental and plant controlling components that needs to grow a plant properly in a green house.

- Environmental conditions
 - Temperature
 - Light level
 - Humidity
 - CO₂
- Plant conditions
 - pH
 - Moisture level
 - Nutrient concentration

In the system All the data of those conditions are collected via sensors which connected inside a greenhouse and all those details are passing to a responsive web app and a web server. The user can control and observe all the environmental and plant controlling functions of his greenhouse via that web app. The web server is used to share the sensor data among sensors.

The responsive web app is the only graphical user interface that interact with the user in this system. Following facts are the major functions that provide to the user from the system web app.

- Get the sensors data and the web server's data in the system to save them in a database and display those details from the user account as log and chart views.
- Sending alert messages to the user about the critical conditions.
- Giving progress reports about the system controlling by comparing them with the pass reports.
- Giving warning reports by comparing the condition statuses with standard values they must use to be.
- Provide tips according to the warning report to the user.
- Can also control all the automated condition controlling that done via the web server

1.3 Goal

Our goal is to control the environmental and plant conditions for a plant to grow healthy. The internet based smart system will control those conditions on its own, anyway it's the responsibility of the user, changing or modifying the environment as it desires.

1.4 Objectives

The Objective of the proposed project is mainly based using the concept of IoT and design automated greenhouse environment and plant condition controller. When you design a system, from it can manage environment and plant condition control data in greenhouse sent over through a wireless network using a web server.

A web server data can be transmitted to the sub system which control environment and plant control condition in the greenhouse.

Prepare the responsive web app, users get the privilege to control automated actions and aware about them. It means prepare a program in this web app.

Chapter 2 – Literature Survey

2.1 Design and implementation of the smart greenhouse environment and plant condition controlling system

This paper proposes an efficient automated plant and environment condition controlling and irrigation system based on computing different changes regarding necessity of greenhouse can be control and monitoring, using wireless network, server, and client web service

There are two main factors in our model to reducing over long-distance power controlling and monitoring. To use various monitoring systems for each plant, about the diverse environmental situations. Not only that the results make out that the control model stands the sensing data and substantial tool for calculation of values in connecting sensors but also output plugged devices which can be use as the self-control.

The purpose of variation of monitoring for automation by using input parameter because the system based on responsive web app.

Results clearly shows us there are so many advantages characteristics that our system like ease of network management and control motors and values.

The parameters we use as input and output in greenhouse control were displayed within the responsive web app. Finally, the process of input and output parameters system measured, and information was given about the rule base.

In this proposed design system, we used five sensors like soil moisture, humidity, temperature, CO₂ and light sensor. Each sensor measures the measure changes of the environment inside the green house.

2.2 IOT based smart greenhouse environment and plant condition controlling system

Automatic drip irrigation used the field of agricultural sector to operate the soil moisture. So, they need limited amount of water is applied to the plants. We concern more about soil health because soil needs N, P, K and any other minerals. Therefore, using drip for irrigation, applying these minerals should be essential.

Checking water level using an ultrasonic sensor and filled the tank. Because water management is very important here. During the night using growing lights land also provide requisite wavelength light to the plants. Using humidity and temperature sensors to control the air humidity, temperature and fogger.

A concept of smart greenhouse, automatically they can go on their work in a farm without less use of manual inspection. Storage containers collected readings and it uploaded to cloud service then it forwarded to an e commerce company.

2.3 IOT based greenhouse using Arduino

Data asset is the most important part in green house because its effect on its climate design had been aimed data asset it. We use multiple sensors to develop data processing for better growth in greenhouse.

We can use firmware of Arduino as software and Arduino board and sensors as hardware, according to the GUI used, through4

It's so easy to read temperature, humidity, CO2 gas using Arduino Nano board, it provides multiple inputs analogues, and I10 digitals through data sensor.

Measuring the soil moisture is so important for irrigation plants, not only that we should applied the intensity of lights in greenhouse. We used microcontroller board. It based on the ATmega328.

These factors help the increase in growth of plants. The purpose of this system is providing and giving ability to control climate of greenhouse.

2.4 Technical review

Arduino is an open-source Programming organization, venture, client group and PC equipment that outlines and produces. The Arduino microcontroller board in light of the ATmega328 is Arduino Uno, It has 6 input, 14 advanced info/yield pins, a 16 MHZ precious stone oscillator, a control jack, a USB association, a reset catch and an ICSP header.

UNO never utilize chip driver FIDI USB, but it contrasts every single before going to board. UNO means one in Italy. It's nominated this because of the new arrival of Arduino 1.0. The most recent progress able one of USB Arduino sheets and reference demonstrate for Arduino stage is "UNO". By using a USB Allocation or with an outer power supply the Arduino UNO Can be powered up. AC to DC connector or battery power can be used as the outer power supply.

Soil Sensor (KG003) is used to detect the moisture level of the soil or judge if there is water around the sensor. The module must insert int the soil and then the on-board potentiometer must adjust to adjust the sensitivity of the sensor. When the moisture is higher/lower according to the threshold set by the potentiometer the sensor will output logic HIGH/LOW.

Temperature Sensor (DHT11) It is equipped with a high-performance 8-bit microcontroller. In humidity calibration chambering the DHT11 sensors performance accruable calibration. The process of calibration coefficients includes OTP program memory, and internal sensors detect signals. To make quick and easy, the single-wire serial interface system was also integrated. The features of the demanding of DHT11 sensors are low power, enabling a variety of applications. Signal can transmit 20 meters distance, small size, according to users need, the sensor can be provided convenient connection and special packages.

Wi-Fi Module (ESP8266) It is a Wi-Fi microchip with full TCP/IP Stack. It also has a microcontroller capacity created by a Shanghai based Chinese maker, Espressif system.

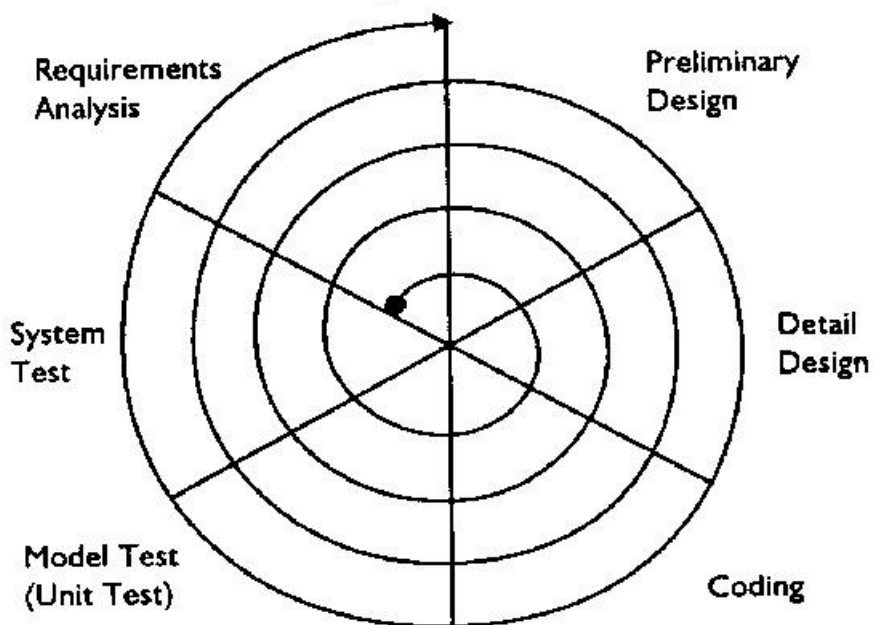
Chapter 3 – Project Planning

3.1 Assumptions & constraints

Assumption means data upload on server and receive the data from web application. It means controlling system.

3.2 Project management approach

Combination of a waterfall and iterative model is known Spiral Model. Version control can be done by this model. It has phase. Phase begins with a goal. At the end the progress is its client reviewing.



3.3 Project Budget

No.	Name of Product	Quantity	Cost / Rs.
1	Ultrasonic sensor	1	180
2	Jumper wires	30	210
3	NPN Transistor	1	8
4	LED	1	4
5	Resistor	2	2
6	Potentiometer	1	25
7	Relay module	2	50
8	Nano board	2	1000
9	Bread board	3	1800
10	Soil moisture sensor	1	190
11	L293D Motor Drive	1	470
12	MQ7 Gas sensor	1	20
13	Fan	1	150
14	DHT11	1	100
15	Water pump	1	150
16	ldr	1	10
17	Environment	1	1300

Chapter 4 – Software Requirements Specification

4.1 Overall description

4.1.1 Product perspective

Greenhouse concept introduce in Sri Lankan agriculture and horticulture. Recently, it becomes important part of this sector today. Plants grow faster in any season and climate is not a big problem to plant growth. Primary stage of this project given up ecological conditions like overheat and less dampness.

Requirements be full fill the plants by automatic monitoring gives a great help to our farmers. By using an artificial condition create required environment conditions at those places.

Purpose of these project is helping the farmers even in opposite environmental conditions. Opposite environment conditions mean conditions of fewer humidity and overheat. Arduino handled the controlling part and IoT handled the monitoring part to check the climatic change in environment. We proposed it needs four main sensors it's system.

We proposed following sensors in the project like light, temperature, humidity and soil sensors. To operate on these sensors, detect availability of light, water in the environment we must be use Arduino UNO and Arduino NANO.

Anywhere any time we have to access the monitoring of system by the web app. It is an open source product, implemented by hardware and software based an application provided by main features.

4.1.2 Product Features

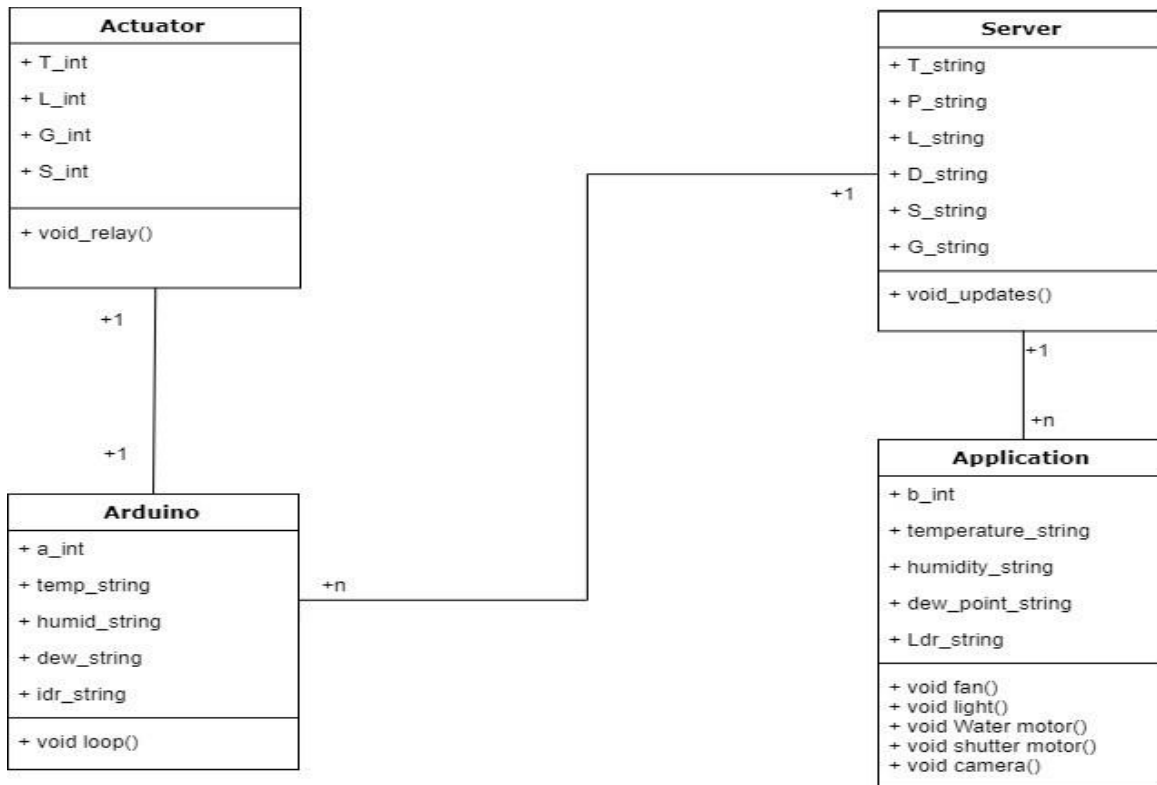
Where ever having net access on any web app, availability of observing of framework done by IoT. It is an open source item. It is execution-based programming and equipment. An application accompanied fundamental component. The products consist two parts. It's taken care by Arduino and the checking part dealt by IoT.

4.1.3 Operating Environment

Requirement of the operating environment for a client and server GUI as we use a Web app, thereafter customer needs working condition and server GUI. It is work with associated working condition.

- Mainly the software depends on the internet, the application can't be work without Wi-Fi.
- Mainly our project depends on hardware

4.1.4 Class diagrams and Characteristics



To control the environment, we can use Arduino UNO sending and receiving data via internet can be used ESP8266. Sending and receiving data to our thingspeak server. Sending and getting information process is done via web used by ESP8266.

4.1.5 Design and Implementation constraints

- Comparing lower speed with wired network.
- More complex to be put in a particular form than wired network.
- Receive abstracts by different elements like Bluetooth.
- It is not easier in buildings make sensing quantities.
- The costs do not reduce, when installation of sensors.

4.2 System Features

- Less expensive in cost.
- Getting information and controlling the environment by Web application.
- Effective Framework.
- Vast amount of automatic control.
- Monitoring and entering of information.
- Due to human error, system eliminating be risk of green house, not be maintained set out environmental conditionals.
- When maintaining a greenhouse associated with less labour costs.
- Customers has ability to define conditions of particular greenhouse.

4.2.1 Description and Priority

No.	Features	Priority	Risk
1	Farmers can ability to monitoring and entering of our system.	10	10
2	Farmers ca be minimized labour cost with the help of IoT technology.	7	2
3	Due to human error, system eliminates be risk of greenhouse, not be maintained Setout environmental conditionals	8	5
4	Farmers have ability to define conditions of particular greenhouse	9	4

4.3 Functional requirements

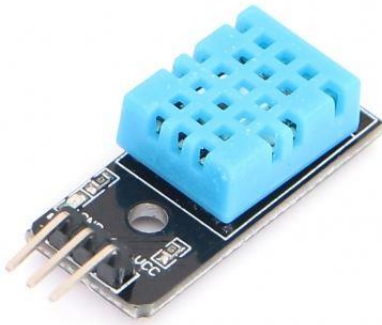
- The temperature should be sense to the sensor, in the given environment of greenhouse system.
- Fetching the correct details of the sensors by the server.
- Response should be given quickly by the server.
- The output be displayed in the screen by immediately

4.4 External Interface Requirements

4.4.1 User Interfaces

We have aimed farmers are the mainly audience of our system. We believe interface should be easily understandable and it is also user friendly to them. In this interface, merged up all the transactions and requirements. The whole system always tried to provide the best utility to the user through this application.

4.4.2 Hardware Interfaces (Sensors)



DHT11 sensor is a Humidity Temperature Sensor. Calibrated digital outputs can be generated by it. DHT11 can be interconnected with any microcontroller like Raspberry Pi, Node MCU, etc. and get ephemeral results.



Soil sensor (KG003) This sensor is the easiest digital method to measure the soil moisture or the water level content around it. Just inserting the sensor into the soil is the only thing that need to do. It will give the moisture level / water level is high when 5V and the moisture level / water level is low when 0V according to a digital output.



As a major aspect of a wellbeing frame work, in a gas sensor, clearly identify, regularly closest place in gases. This type of gear can recognize different discharges ad a gas spill. As a result, interface can be used as a control framework, and done naturally close process. Where the break is happening, a gas indicator makes a sound to alert knowing the administrators in the relevant region.



An LDR is another type of variable light-controlled resistor. There is another resistance here It known as photo resistor. It acts here as an exhibits photo conductivity or by the way it decreases rising incident light intensity, it simply means defects presence of light in the environment.



Wi-Fi module (ESP8266) interconnects all the sensors and Arduino with the internet to web application. This sensor should be updates like, information in database and both side information in the thinkspeak server.

The Arduino controller can be used usually small scale of application like Greenhouse, etc. The sensors we use to this project, sensing variable reaction from the environment, as its availability. Like humidity and temperature, moisture of the soil, CO2 in PPM, etc. The activation of actuators decided. Passed the sense values to Arduino ATmega328 work as a micro controller board known as Arduino UNO an on it Out of the 14 digital i/o pins its six are analogue inputs.

For an example, Arduino can activate fan, here these sensors pass the message to the Arduino when there is no wind in agriculture. It provides wind connected of internet cloud via Wi-Fi. It can be stored activated or not continue data by actuators.

If the server is down or Wi-Fi is unable to build up the connection with the cloud, a web app auto updated a server down message is printed else data.

4.5 Non-functional Requirements

4.5.1 Performance Requirements

- **Speed of data receiving:** the wrong solution can cause more delay, because the sensor should be more speed able, as a result of speed of retrieving speed.
- **Speed of data manipulating:** It may lead to miss, given database should be faster and fetched data be manipulating.
- **Sending Notification:** Time should be normal in notification; therefore, farmer can fix the problem and know about the complete situation.
- **Display timing:** Display time is the most important part to the farmer in this process. Though the farmer should not get the information if display not work properly by the way it is his duty select which crop is choose next.

4.5.2 Safety Requirements

- **Use case:** We use use-case to maintenance of the core circuit. It consists Arduino Wi-Fi module, sensors, wires, etc. and Safe guard from external affairs. Because of that we use it external safe guard avoid the disturbances of the circuit of the system.
- **IoT:** It consists all the parameters relevant to the Internet such as http, https, communication protocols and Wi-Fi protocols. IoT means Internet of things.
- **Database:** Secure of the Database is important. Because it connects information to farmer, farm manager, crop details and other reasons. Otherwise a little bit of changes in database can cause server fault of the system by avoiding information to get another hand.
- **Web app:** code must be so personalized to developer in web application. Because anyone can change components, features, application functionalists that related to the environment.

4.5.3 Security Requirements

- **Availability:** Any part of the earth, system should be available. According with internet connection that supportive IoT.
- **Correctness:** Send and receive data must be accurate and correct regularly.
- **Flexibility:** The system should be work with a web app.
- **Reliability:** According to request of the farmer, the system should be activated.
- **Maintainability:** The system can be done easily, as it doesn't enough maintenance.

Chapter 5 – System Design

5.1 System Requirement Definition

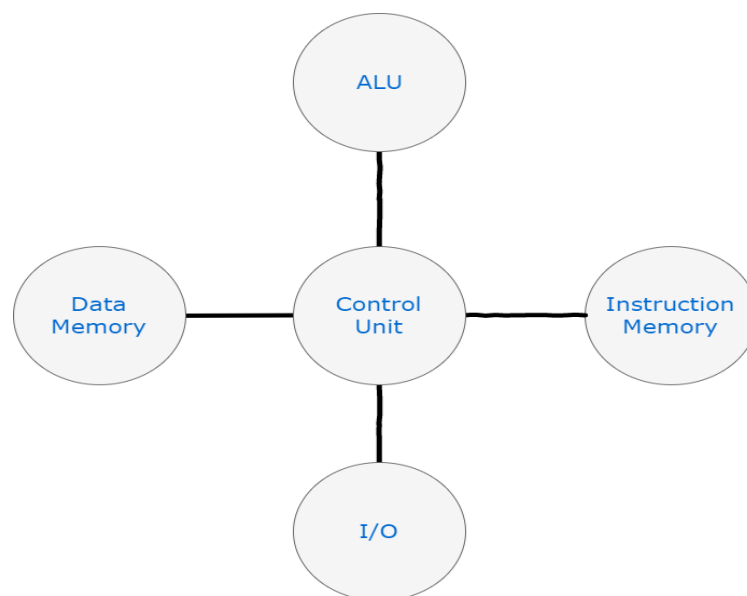
It is an open source development platform based on Software and Hardware known as Arduino. It is easy to use Arduino micro controller even in powerful single board computer and compilation. Arduino boards, programming language and software for development based on combination of micro controller.

The Arduino connect with all the sensors and acualators. We join the Arduino to the internet using Wi-Fi module. An interface will be provided by cloud server between the user and the database.

For an example AWS, Thingspeak using board and fixed the Arduino shields. We can use ESP8266, GSM module and ZigBee except Arduino shields.

Zigbee devices can be used to reach long distance via internet. It used as mesh network of complex devices. The defined rate of ZigBee has 250 Kbits/s, It is best for default data transmissions from a sensor or input device sending through a message or miscall via network, We use GSM modules. The cheap Wi-Fi chip is ESP8266 and load full of TCPIIP stack.

Transceiver can be added as existing micro controller based set up of this, enable the system communication with Internet via Wi-Fi. Through the internet, command will be sent from a web application. It sends Wi-Fi network to ESP8266 with Arduino UNO. We can save our data storing on Google drive.

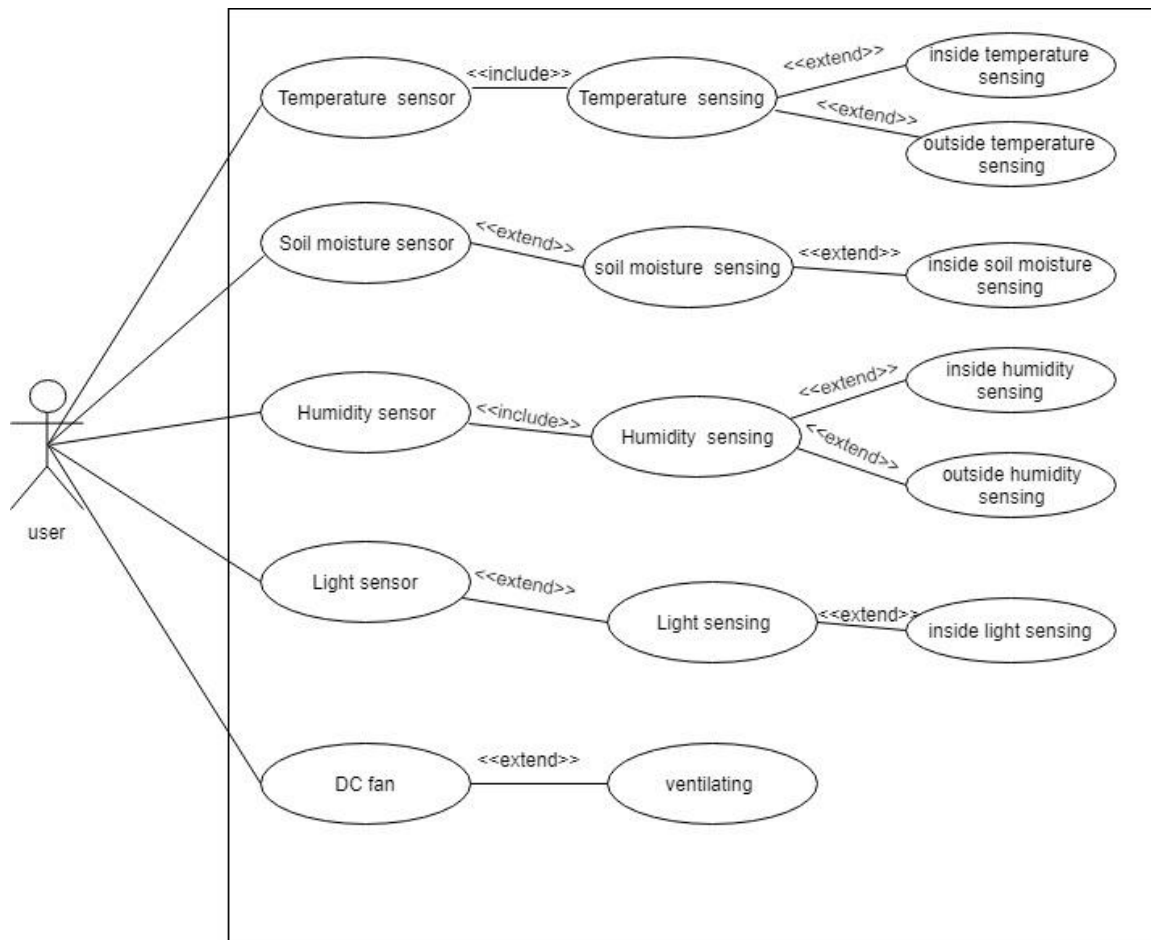


5.2 Functional requirements

Arduino associated with all the sensors and actuators. A Wi-Fi module can be used to take part the Arduino to the internet. An interface between the client and the database will be given by Cloud server. Reasons for that Cloud server are AWS, Thingspeak settle on board we can use Arduino shields. Through ESP8266, ZigBee and GSM module use remotely too.

5.3 Use case Diagram

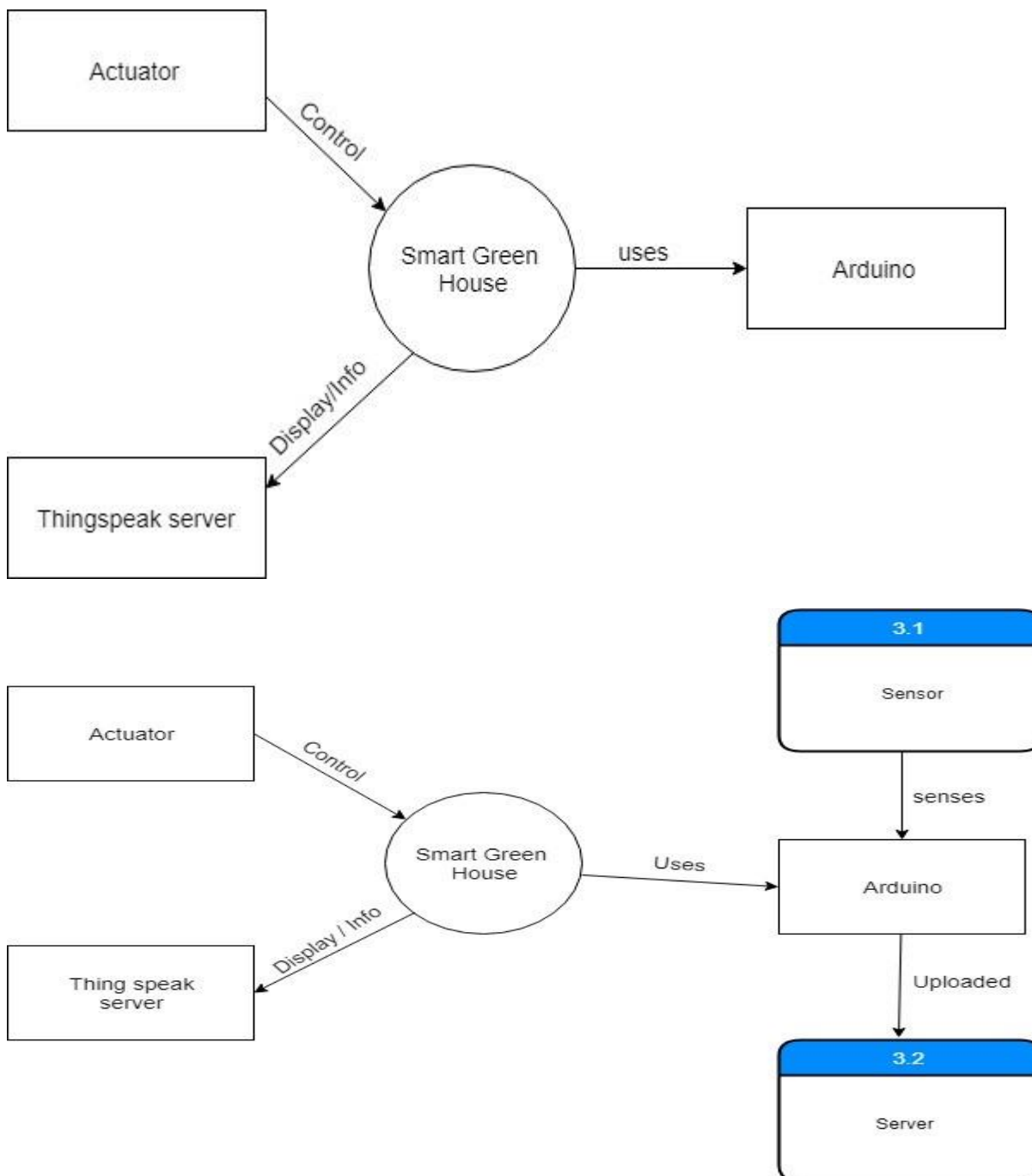
Viewing the notifications, performing a manual action can monitoring in the green house by using the use case diagram. The diagram displays the basic needs of the smart green house.

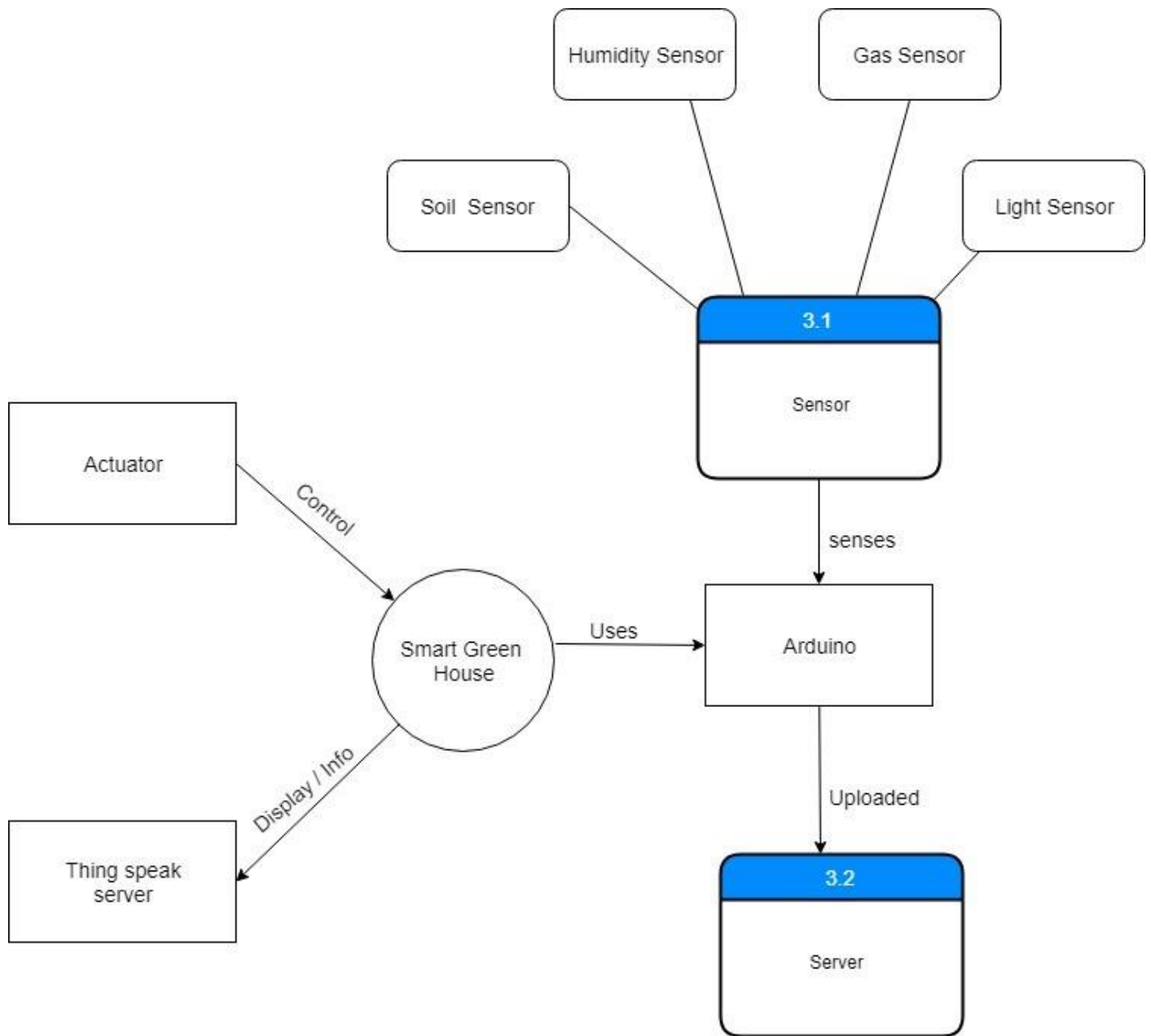


5.4 Data Flow Diagram

The flow of information for any process or system coming out of the diagram. It uses defined symbols to show data inputs, outputs, storage points and the routes between each destination. Symbols they like to use circles, rectangles, arrows, plus short text labels. It is simple, hand – drawn process shows overview depth. Deeper can see progressively in multilevel DFD's and it represent how to handle the data.

Analysing system or model a new one, we can be used this. DFD diagrams and charts regularly try to explain words, both technical and nontechnical audiences work to CEO from developer. As a reason it becomes more popular all these years. They work well because of the data flow software and systems. Those days it is less applicable because of graphical user interface friendly, real time or database-oriented software or systems.





5.5 Sub-system Development

The system basically divided in many parts, but hardware and software are the two main parts. Again, Hardware divided in two sections like sensors and transmission of data. Explain of modules are follows.

5.5.1 Sensor testing

We try our best to check all the sensors. Each sensor we connected individually to the Arduino. For an example testing for environment humidity, temperature, dew point in the atmosphere. We can use DHT11. The arrangement of the pin of Arduino UNO and DHT11 understood us.

We accompanied each sensor freedom to the Arduino accompanied with DHT11, we have to test slickness, temperature, dew point in the climate of the location. The pin layout of Arduino UNO and DHT11 be make a sense of us. Each sensor should relate independently to Arduino.

There are special files should be downloaded to the DHT11 to run it. Not only that we should set com port for Arduino to identify it our laptop.

5.5.2 Wi-Fi Connectivity

By sending sensors value to our server thingspeak. We should test ESP8266. In this module, to connect our Arduino to the internet we used Wi-Fi module ESP8266. Our server can get data from it. We always try to connect each sensor a partly to the server individually in this module.

Module No.2 we used to send the sensor values to our server thingspeak. To connect the Arduino with the internet we used Wi-Fi module ESP8266. Our server can get data from it.

5.5.3 Status Shown

Third module represent phpMyAdmin data about the sensors. The duty of this sensor by, data fetching. Fetch from the respective servers. This module be inter-connected into one app using the Web functional integrity.

5.5.4 Joining all together

While the sensors uploading the value to our server, all the sensors be interconnected. As our server we use thinkspeak and shields for our sensor.

5.6 Systems Integration

When we analyse the First module it describes connecting each sensor separately to the Arduino, its pin configuration of DHT11 and Arduino. Second module about i.e. thingspeak we used to send the sensor values to our server. There we used Wi-Fi module ESP8266 to connect our Arduino to the internet then it can send data to our server.

Third module is about data fetching about phpMyAdmin data about the sensors, fetch from the respective servers. All these three modules are interconnected with one web app.

Chapter 6 – Implementation

6.1 Sensors module

```
#define VISUINO_ARDUINO_NANO

#include <OpenWire.h>
#include <Mitov.h>
#include <Mitov_FormattedSerial.h>
#include <Mitov_DHT_Sensor.h>
#include <Mitov_Text.h>

// Arduino Board Declarations

namespace BoardDeclarations
{
Mitov::TypedSerialPort<Mitov::SerialPort<SERIAL_TYPE, Serial>, SERIAL_TYPE, Serial> SerialPort0;
Mitov::ArduinoSerialStringInput<Mitov::TypedSerialPort<Mitov::SerialPort<SERIAL_TYPE, Serial>, SERIAL_TYPE, Serial>,
SerialPort0> SerialPort0_String_1;
Mitov::ArduinoDigitalInputChannel<2, false> ArduinoDigitalChannel_2( false );
} // BoardDeclarations

// Interrupts

namespace Interrupts
{
void Handler2();

OpenWire::PinFallingInterrupt Pin2( 2, Handler2 );

void Handler2()
{
Pin2.InterruptHandler();
}

} // Interrupts

// Declarations

namespace Declarations
{
Mitov::DHT11Sensor_Interrupt<2, Mitov::AnalogPin_StandardPinImplementation<2>,
Mitov::AnalogPin_StandardPinImplementation<3>, Interrupts::Handler2> HumidityThermometer1;
Mitov::FormattedText_Fixed<2> FormattedText1;
Mitov::FormattedTextElementAnalog<Mitov::FormattedText_Fixed<2>, Declarations::FormattedText1>
TArduinoFormattedTextElementAnalog1;
Mitov::FormattedTextElementAnalog<Mitov::FormattedText_Fixed<2>, Declarations::FormattedText1>
TArduinoFormattedTextElementAnalog2;
} // Declarations

// Call Chains

namespace CallChains
{
void GetText1(uint32_t AIndex, uint8_t AOperation, bool &ABoolResult, Mitov::String &AResult)
{
Declarations::TArduinoFormattedTextElementAnalog1.GetText( 0, AIndex, AOperation, ABoolResult, AResult );
Declarations::TArduinoFormattedTextElementAnalog2.GetText( 1, AIndex, AOperation, ABoolResult, AResult );
}

} // CallChains

// Pin Call Declarations

namespace PinCalls
{
void PinCallerReceive1( void * _Data );
```

```

void PinCallerReceive2( void *_Data );
void PinCallerReceive3( void *_Data );
} // PinCalls

// Pin Call Implementations

namespace PinCalls
{
void PinCallerReceive1( void *_Data )
{
Declarations::TArduinoFormattedTextElementAnalog1.InputPin_o_Receive( _Data );
}

void PinCallerReceive2( void *_Data )
{
Declarations::TArduinoFormattedTextElementAnalog2.InputPin_o_Receive( _Data );
}

void PinCallerReceive3( void *_Data )
{
BoardDeclarations::SerialPort0_String_1.InputPin_o_Receive( _Data );
}
} // PinCalls

namespace ComponentsHardware
{
void SystemUpdateHardware()
{
}
} // ComponentsHardware

//The setup function is called once at startup of the sketch
void setup()
{
Interrupts::Pin2.SubscribeFalling( (OpenWire::Object *)&Declarations::HumidityThermometer1, (OpenWire::TONInterrupt)( void (OpenWire::Object::*)( bool ) )&Mitov::DHT11Sensor_Interrupt<2, Mitov::AnalogPin_StandardPinImplementation<2>, Mitov::AnalogPin_StandardPinImplementation<3>, Interrupts::Handler2>::InterruptHandler );
Declarations::FormattedText1.Elements_CallChain = ::CallChains::GetText1;
Declarations::FormattedText1.AddReadyElement( (char *)"Temperature: ", 0 );
Declarations::FormattedText1.AddReadyElement( (char *)" Humidity: ", 1 );

Declarations::HumidityThermometer1.TemperatureOutputPin().SetCallback( PinCalls::PinCallerReceive1 );
Declarations::HumidityThermometer1.HumidityOutputPin().SetCallback( PinCalls::PinCallerReceive2 );
Declarations::FormattedText1.OutputPin().SetCallback( PinCalls::PinCallerReceive3 );

BoardDeclarations::SerialPort0.SystemInit();
Declarations::HumidityThermometer1.SystemInit();

Declarations::HumidityThermometer1.SystemStart();
Declarations::FormattedText1.SystemStart();
Declarations::TArduinoFormattedTextElementAnalog1.SystemStart();
Declarations::TArduinoFormattedTextElementAnalog2.SystemStart();

OpenWire::SystemStarted();
}

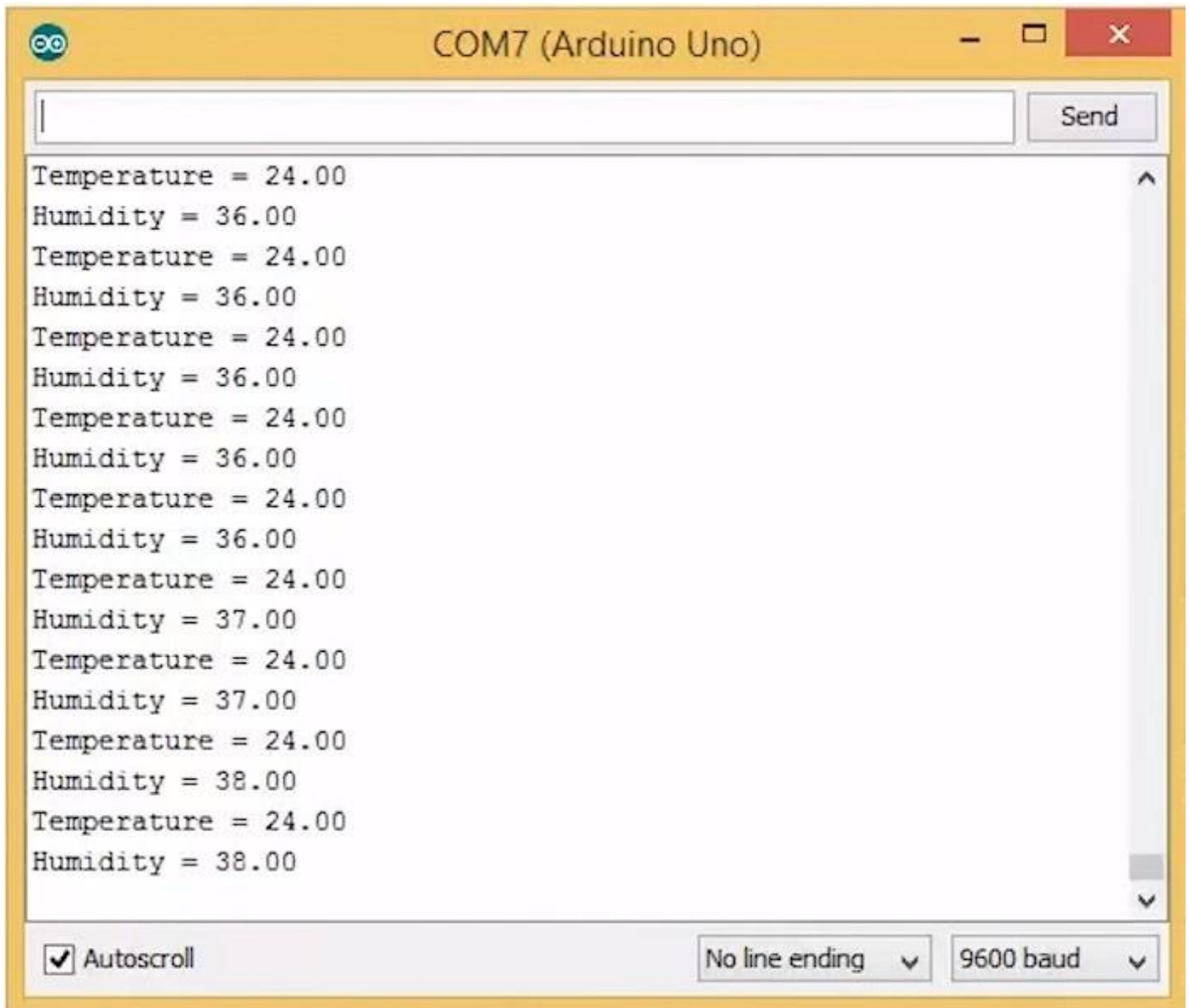
// The loop function is called in an endless loop
void loop()
{
unsigned long A_Current_Microseconds = micros();

BoardDeclarations::SerialPort0.SystemLoopBegin( A_Current_Microseconds );
Declarations::HumidityThermometer1.SystemLoopBegin( A_Current_Microseconds );

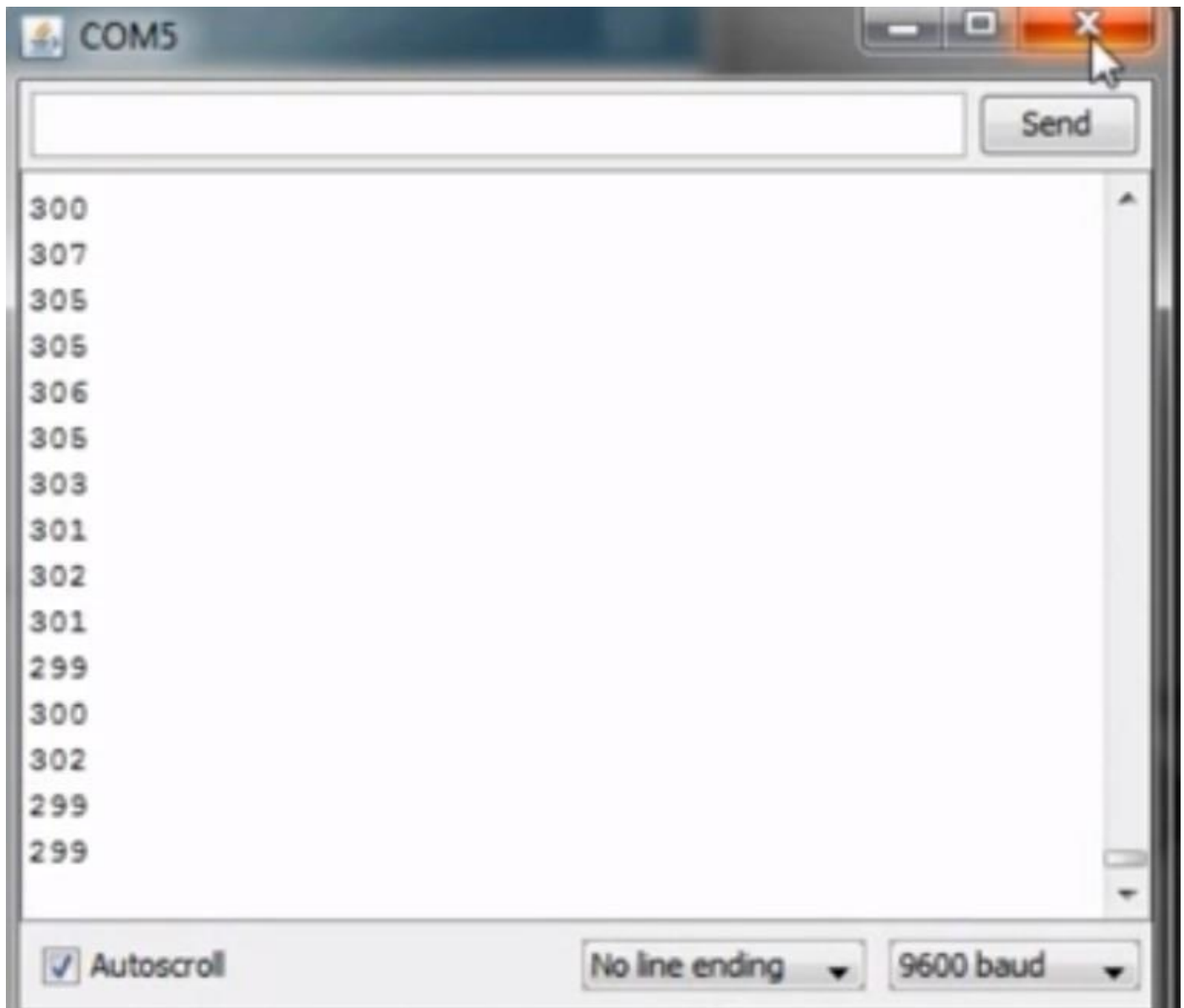
Declarations::FormattedText1.SystemLoopEnd( A_Current_Microseconds );
}

```

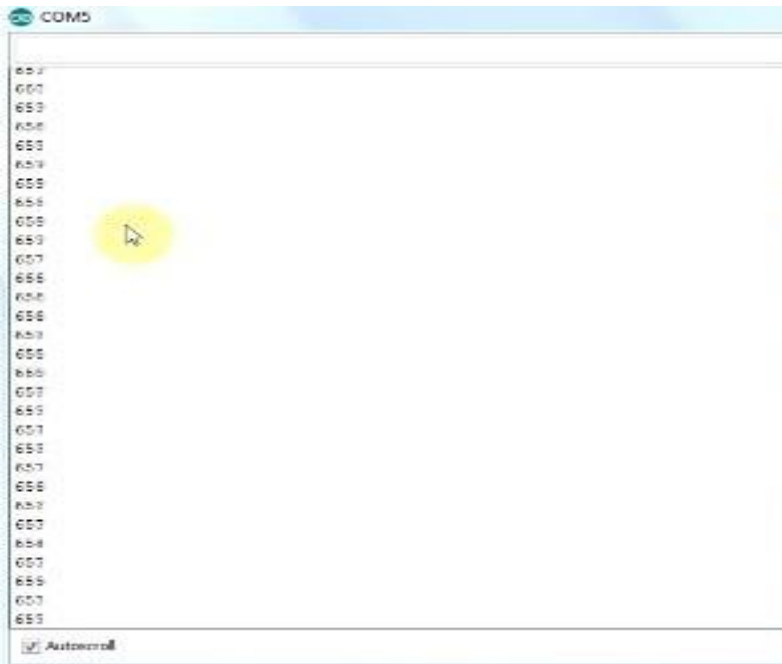
Serial monitor for DHT11



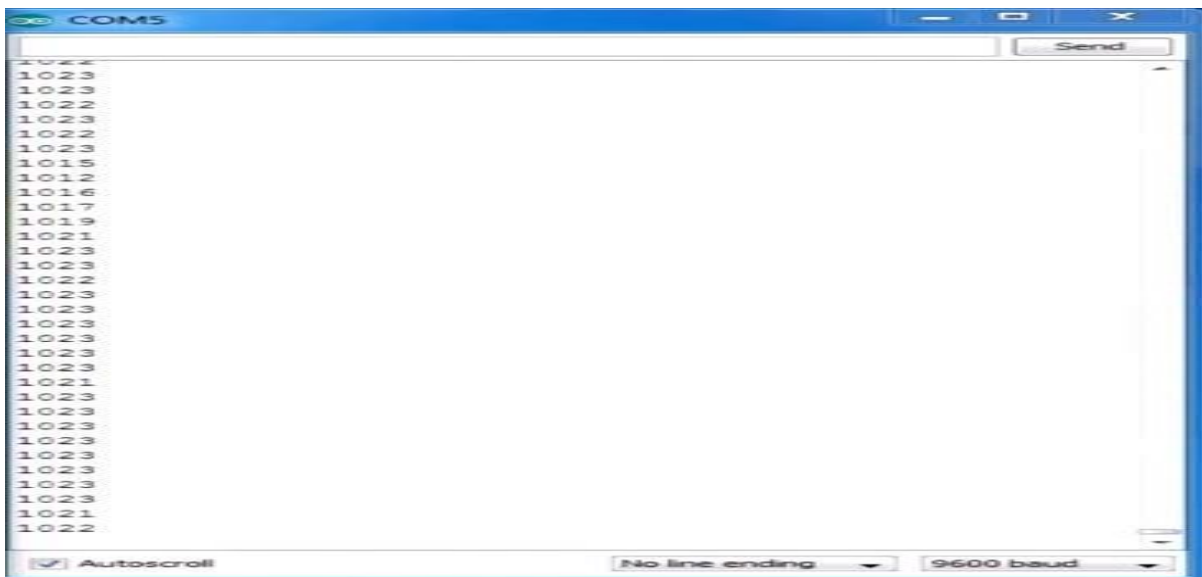
Monitor for LDR



Serial monitor for Gas



Serial monitor for soil sensor



6.2 Wi-Fi connectivity

Temperature CSV file

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
70	2017-02-07 15:15:14 UTC	549	33																			
71	2017-02-07 15:15:39 UTC	550	33																			
72	2017-02-07 15:16:05 UTC	551	33																			
73	2017-02-07 15:16:30 UTC	552	33																			
74	2017-02-07 15:16:56 UTC	553	33																			
75	2017-02-07 15:17:21 UTC	554	33																			
76	2017-02-07 15:17:46 UTC	555	33																			
77	2017-02-07 15:18:12 UTC	556	32																			
78	2017-02-07 15:18:37 UTC	557	33																			
79	2017-02-07 15:19:03 UTC	558	33																			
80	2017-02-07 15:19:30 UTC	559	33																			
81	2017-02-07 15:19:54 UTC	560	127																			
82	2017-02-07 15:20:19 UTC	561	33																			
83	2017-02-07 15:20:45 UTC	562	33																			
84	2017-02-07 15:21:10 UTC	563	33																			
85	2017-02-07 15:23:53 UTC	564	32																			
86	2017-02-07 15:24:18 UTC	565	33																			
87	2017-02-07 15:24:44 UTC	566	33																			
88	2017-02-07 15:25:09 UTC	567	33																			
89	2017-02-07 15:25:35 UTC	568	33																			
90	2017-02-07 15:26:00 UTC	569	33																			
91	2017-02-07 15:26:28 UTC	570	33																			
92	2017-02-07 15:26:51 UTC	571	33																			
93	2017-02-07 15:27:16 UTC	572	33																			
94	2017-02-07 15:27:42 UTC	573	33																			
95	2017-02-07 15:28:08 UTC	574	33																			
96	2017-02-07 15:28:34 UTC	575	33																			
97	2017-02-07 15:28:59 UTC	576	33																			
98	2017-02-07 15:29:25 UTC	577	33																			

Humidity CSV file

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
72	2018-04-0 551	14																					
73	2018-04-0 552	14																					
74	2018-04-0 553	14																					
75	2018-04-0 554	13																					
76	2018-04-0 555	13																					
77	2018-04-0 556	14																					
78	2018-04-0 557	14																					
79	2018-04-0 558	14																					
80	2018-04-0 559	14																					
81	2018-04-0 560	127																					
82	2018-04-0 561	13																					
83	2018-04-0 562	14																					
84	2018-04-0 563	14																					
85	2018-04-0 564	13																					
86	2018-04-0 565	14																					
87	2018-04-0 566	14																					
88	2018-04-0 567	14																					
89	2018-04-0 568	14																					
90	2018-04-0 569	14																					
91	2018-04-0 570	14																					
92	2018-04-0 571	13																					
93	2018-04-0 572	14																					
94	2018-04-0 573	14																					
95	2018-04-0 574	13																					
96	2018-04-0 575	13																					
97	2018-04-0 576	14																					
98	2018-04-0 577	13																					
99	2018-04-0 578	13																					
100	2018-04-0 579	13																					

Dewpoint CSV file

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
72	2018-04-0	551	1.91																				
73	2018-04-0	552	1.91																				
74	2018-04-0	553	1.91																				
75	2018-04-0	554	0.87																				
76	2018-04-0	555	0.87																				
77	2018-04-0	556	1.12																				
78	2018-04-0	557	1.91																				
79	2018-04-0	558	1.91																				
80	2018-04-0	559	1.91																				
81	2018-04-0	560	134.81																				
82	2018-04-0	561	0.87																				
83	2018-04-0	562	1.91																				
84	2018-04-0	563	1.91																				
85	2018-04-0	564	0.09																				
86	2018-04-0	565	1.91																				
87	2018-04-0	566	1.91																				
88	2018-04-0	567	1.91																				
89	2018-04-0	568	1.91																				
90	2018-04-0	569	1.91																				
91	2018-04-0	570	1.91																				
92	2018-04-0	571	0.87																				
93	2018-04-0	572	1.91																				
94	2018-04-0	573	1.91																				
95	2018-04-0	574	0.87																				
96	2018-04-0	575	0.87																				
97	2018-04-0	576	1.91																				
98	2018-04-0	577	0.87																				
99	2018-04-0	578	0.87																				
100	2018-04-0	579	0.87																				

6.3 Connecting to Thing speak Server

```

\begin{figure}[htbp!]
\centering
\includegraphics[height=09cm,width=15cm]{project/images/ScreenShorts/allsen1
.png}

\caption{Screenshot of Thinkspeak graph 1}
5 \label{fig:Thinkspeak graph 1}
6\end{figure}
7\begin{figure}[htbp!]
8\centering
9 \includegraphics[height=09cm,width=15cm]{project/images/Scr
eenShorts/Allsens2.png}
10 \caption{Screenshot of Thinkspeak graph 2}
11 \label{fig:Thinkspeak graph 2}
12\end{figure}
13\begin{figure}[htbp!]
14\centering
15 \includegraphics[height=09cm,width=15cm]{project/images/ScreenShorts/Allse
n3

```

```

        . png }
16 \caption{ Screenshot of Thinkspeak graph 3}
17 \label{fig: Thinkspeak graph 3}
18 \end { figure }
19
20
21 \begin { Istlisting }
22 #include < SoftwareSerial.h> //includethesoftwareseriallibrary
23 #include <dht.h> //includethedhtlibrary
24 dht DHT; //initiatetheDHTobject
25 int a; //avariabletostoretherawsensorvalue
26 String temp; //Variabletostorethetemperature
27 String humid; //Variabletostorethehumidity
28 String dew; //Variabletostorethedewpoint
29 String ldr; //Variabletostoretheldrvalue
30 String soil; //Variabletostorethesoilsensorvalue
31 String gas; //Variabletostorethegassensorvalue
32 SoftwareSerial esp8266(3,4); //set the software serial pins RX pin =
    3, TX pin =4
33 //definitionofvariables
34 #define DEBUG true //show messages between ESP8266 and Arduino in se
    rial port ,when settotru e
35 #define SSID "Arzaan" //replacexwithyourwifinetworkname
36 #define PASS "alohomora" //replacexwithyourwifinetworkpassword
37 String sendAT (String command , const int timeout , boolean debug)
38 {
39     String response = "";
40     esp8266 . print (command);
41     long int time = millis ();
42     while ( (time + timeout) > millis ())
43     {
44         while (esp8266 . available ())
45         {
46             char c = esp8266 . read ();
47             response += c;
48         }
49     }
50     if (debug)
51     {
52         Serial . print (response);
53     }

```

```

54     return response;
55 }
56 void setup()
57 {
58     Serial.begin(115200); // begin the serial communication with baud of 9600
59     esp8266.begin(115200); // begin the software serial communication
        with baudrate 9600
60     sendAT("AT+RST\r\n", 2000, DEBUG); // call sendAT function to send rese
        t AT command
61     sendAT("AT\r\n", 1000, DEBUG);
62     sendAT("AT+CWMODE=1\r\n", 1000, DEBUG); // call sendAT function to set
        ESP8266 to station mode
63     sendAT("AT+CWLAP=\""SSID"\",\"\"PASS\""\r\n", 2000, DEBUG); // AT
        command to connect with the wifi network
64     while(!esp8266.find("OK")) { // wait for connection
65     }
66     sendAT("AT+CIFSR\r\n", 1000, DEBUG); // AT command to print IP ad
        dress on serial monitor
67     sendAT("AT+CIPMUX=0\r\n", 1000, DEBUG); // AT command to set
        ESP8266 to multiple connections
68 }
69 void loop() {
70     /// put your main code here , to run repeatedly :
71     a=DHT.read11(5); // read the sensor data and store it in variable a
72     temp=DHT.temperature; // store the temperature values in temp variable
73     humid=DHT.humidity; // store the humidity values in humid variable
74     double gamma = log(DHT.humidity / 100) + ((17.62 * DHT.temperature) / (
        243.5 + DHT.temperature));
75     double dp = 243.5 * gamma / (17.62 - gamma);
76     d/ew=(String) dp;
77     ldr=digitalRead(6);
78     soil=digitalRead(7);
79     gas=digitalRead(8);
80     Serial.print("Temperature:"); // print the temperature on serial monitor
81     Serial.println(temp); // Print the temperature value
82     Serial.print("Humidity:"); // Print Humidity on serial monitor
83     Serial.print(humid); // Print the humidity value on serial monitor
84     Serial.print("Dew : "); // Print Dew : on serial monitor
85     Serial.print(dew); // Print the dew point value on serial monitor
86     Serial.print("LDR :");
87     Serial.print(ldr); // Print the LDR value on serial monitor

```

```

88 Serial.print("soil sensor:");
89 Serial.print(soil); // Print the Soil sensor value on serial monitor
90 Serial.print("gas sensor:");
91 Serial.print(gas); // Print the gas sensor value on serial monitor
92 Serial.print("\n");
93 updateTS(temp, humid, dew, ldr, soil, gas); // call the function to
    o update Thingspeak channel
94 delay(3000);
95 }
96 void updateTS(String T, String P, String D, String L, String S, String G) {
97     Serial.println("");
98     sendAT("AT+CIPSTART=\"TCP\", \"api.thingspeak.com\", 80\r\n", 500, DEBUG);
99     delay(1000);
100    String cmdlen;
101    String cmd="GET /update?key=YSBBBRD7OBT9XQG9&field1="+T+"&field2="+P+"&f
        ield3="+D+"&field4="+L+"&field5="+S+"&field6="+G+"\r\n"; // update the
        temprature and humidity values on thingspeak url, replace xxxxxxxx
        with your write api key
102    cmdlen = cmd.length();
103    sendAT("AT+CIPSEND="+cmdlen+"\r\n", 1000, DEBUG);
104    esp8266.print(cmd);
105    Serial.println("");
106    sendAT("AT+CIPCLOSE\r\n", 1000, DEBUG);
107    Serial.println("");
108    delay(1000);
109 }

```

6.4 Communication between hardware & software

```
1#include <SoftwareSerial.h> //includethesoftwareseriallibrary
2#include <Servo.h>
3#include <dht.h> //includethedhtlibrary
4dhtDHT; //initiatetheDHTobject
5inta; //avariabletostoretherawsensorvalue
6String temp; //Variabletostorethetemperature
7String humid; //Variabletostorethehumidity
8String dew; //Variabletostorethedewpoint
9Stringldr; //Variabletostoretheldrvalue
10Stringsoil; //Variabletostorethesoilsensorvalue
11Stringgas; //Variabletostorethegassensorvalue
12 SoftwareSerial esp8266(3,4); //set the software serial pins RX pin =
    3, TX pin =4
13//definitionofvariables
14#define DEBUG true //show messages between ESP8266 and Arduino in se
    rial port ,when settotru e
15#define SSID "Arzaan" //replacexwithyourwifinetworkname
16#define PASS "alohomora" //replacexwithyourwifinetworkpassword
17String sendAT (String command , constint timeout , boolean debug)
18{
19    String response = "";
20    esp8266 . print (command);
21    long int time = millis ();
22    while ( (time + timeout) > millis ())
23    {
24        while (esp8266 . available ())
25        {
26            char c = esp8266 . read ();
27            response += c;
28        }
29    }
30    if (debug)
31    {
32        Serial . print (response);
33    }
34    return response;
35}
36//-----Relayinitialization
37int tempR; //integerfortemprelay
```

```

38 intldrR;//integerforldrrelay
39 intsoilR;//integerforsoilrelay
40 intgasR;//integerforgasrelay
41 //----- END
42 Servo left;
43 Servo right;
44 int pos = 0;
45 /*
46 //----- OLED Display
47 #include <Wire.h>
48 #include <AdafruitSSD1306.h>
49 #include <AdafruitGFX.h>
50 Adafruit SSD1306 display(-1);
51 #if (SSD1306_LCDHEIGHT != 64)
52 #error("Height incorrect, please fix AdafruitSSD1306.h!");
53 #endif
54 //----- END8 */
55 void setup()
56 {
57     Serial.begin(115200); // begin the serial communication with baud of 9600
58     esp8266.begin(115200); // begin the software serial communication
        with baudrate 9600
59     sendAT("AT+RST\r\n", 2000, DEBUG); // call sendAT function to send reset AT command
60     sendAT("AT\r\n", 1000, DEBUG);
61     sendAT("AT+CWMODE=1\r\n", 1000, DEBUG); // call sendAT function to set
        ESP8266 to station mode
62     sendAT("AT+CWJAP=\"\"SSID\"\",\"\"PASS\"\"\r\n", 2000, DEBUG); // AT
        command to connect with the wifi network
63     while(! esp8266.find("OK")) { // wait for connection
64     }
65     sendAT("AT+CIFSR\r\n", 1000, DEBUG); // AT command to print IP address on serial monitor
66     sendAT("AT+CIPMUX=0\r\n", 1000, DEBUG); // AT command to set
        ESP8266 to multiple connections
67 //----- Relay setup
68 pinMode(10, OUTPUT);
69 digitalWrite(10, HIGH);
70 pinMode(11, OUTPUT);
71 digitalWrite(11, HIGH);
72 pinMode(12, OUTPUT);

```

```

73  digitalWrite(12, HIGH);
74  pinMode(9, OUTPUT);
75  digitalWrite(9, HIGH);
76  left.attach(7);
77  right.attach(A3);
78  left.write(0);
79  right.write(0);
80  /*//-----OLEDdisplaysetup
81  //initializeandcleardisplay
82  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
83  display.clearDisplay();
84  display.display();
85  //displayalineof text
86  display.setTextSize(1);
87  display.setTextColor(WHITE);*/
88  }
89  void loop() {
90  /// put your main code here , to run repeatedly :
91  a=DHT.read11(5); // read the sensor data and store it in variable a
92  temp=DHT.temperature; // store the temperature values in temp variable
93  tempR= DHT.temperature; // store the temperature value in int
94  humid=DHT.humidity; // store the humidity values in humid variable
95  double gamma = log(DHT.humidity / 100) + ((17.62 * DHT.temperature) / (
    243.5 + DHT.temperature));
96  double dp = 243.5 * gamma / (17.62 - gamma);
97  dew= (String) dp;
98  ldr=digitalRead(6);
99  ldrR=digitalRead(6);
100  soil= analogRead(A2);
101  soilR=analogRead(A2);
102  gas= digitalRead(A0);
103  gasR= digitalRead(A0);
104  Serial.print("Temperature:"); // print the temperature on serial monitor
105  Serial.println(temp); // Print the temperature value
106  Serial.print(" Humidity :"); // Print Humidity on serial monitor
107  Serial.print(humid); // Print the humidity value on serial monitor
108  Serial.print(" Dew : "); //Print Dew : on serial monitor
109  Serial.print(dew); // Print the dew point value on serial monitor
110  Serial.print(" LDR :");
111  Serial.print(ldr); // Print the LDR value on serial monitor
112  Serial.print(" soil sensor :");
113  Serial.print(soil); // Print the Soil sensor value on serial monitor

```

```

114 Serial.print("gas sensor:");
115 Serial.print(gas); // Print the gas sensor value on serial monitor
116 Serial.print("\n");
117 updateTS(temp, humid, dew, ldr, soil, gas); // call the function to
    o update Thingspeak channel
118 Relay(tempR, ldrR, soilR, gasR);
119 // oled(temp, humid, dew, ldr, soil, gas);
120 delay(3000);
121 }
122 void updateTS(String T, String P, String D, String L, String S, String G) {
123     Serial.println("");
124     sendAT("AT+CIPSTART=\"TCP\", \"api.thingspeak.com\", 80\r\n", 1000, DEBUG);
125     delay(2000);
126     String cmdlen;
127     String cmd="GET /update?key=YSBBBRD7OBT9XQG9&field1="+T+"&field2="+P+"&f
        ield3="+D+"&field4="+L+"&field5="+S+"&field6="+G+"\r\n"; // update the
        temprature and humidity values on thingspeak url, replace xxxxxxx
        with your write api key
128     cmdlen = cmd.length();
129     sendAT("AT+CIPSEND="+cmdlen+"\r\n", 2000, DEBUG);
130     esp8266.print(cmd);
131     Serial.println("");
132     sendAT("AT+CIPCLOSE\r\n", 2000, DEBUG);
133     Serial.println("");
134     delay(15000);
135 }
136 void Relay(int T, int L, int S, int G) {
137     digitalWrite(10, HIGH);
138     digitalWrite(11, HIGH);
139     digitalWrite(12, HIGH);
140     digitalWrite(2, HIGH);
141     if(T>=30)
142     {
143         digitalWrite(10, LOW); // set the digital pin 10 off to make Relay
            on for fan
144         delay(100);
145     }
146     else
147     {
148         digitalWrite(10, HIGH); // set the digital pin 10 off to make Relay
            off for fan

```

```

149   delay(100);
150   }
151   if (L== 0)
152   {
153       digitalWrite(11, HIGH);           // sets the digital pin 10 off to make
           Relay on for fan
154       delay(100);
155   }
156   else
157   {
158       digitalWrite(11, LOW);           // sets the digital pin 10 off to make
           Relay off for fan
159       delay(100);
160   }
161   if (S>650)
162   {
163       digitalWrite(12, LOW);           // sets the digital pin 10 off to make
           Relay on for fan
164       delay(100);
165   }
166   else
167   {
168       digitalWrite(12, HIGH);           // sets the digital pin 10 off to make
           Relay off for fan
169       delay(100);
170   }
171   if (G==0)
172   {
173       digitalWrite(9, LOW);           // sets the digital pin 10 off to make R
           elay on for fan
174       for (pos = 0; pos <=90; pos += 1) {
175           left.write(pos);
176           right.write(pos);
177           delay(15);
178       }
179       delay(50);
180   }
181   else
182   {
183       digitalWrite(9, HIGH);           // sets the digital pin 10 off to make
           Relay off for fan

```

```
184 left.write(0);  
  
185     right.write(0);  
  
186     delay(100);  
  
187     }  
  
188 }
```

Chapter 7 – System Testing

7.1 Test Cases and Test Results

Test ID	Test Case Title	Test Condition	System Behaviour	Expected Result
T01	Individual Sensors	The sensors operate separately	Do not work properly Gas sensor mq7	Each sensor operates properly
T02	Server uploading	Sending data to the server through Arduino	Internet be disconnected if something misleads it. As a result, sending data never receive to the internet	Data sending to the internet by sensors
T03	Integration	Sense and sending data via internet to the server by interconnected sensors to the Arduino	Sensing and sending data over internet by all the sensors	Sensing and sending data via internet by all the sensors.
T04	Web App	Responsive web app controls the system	Control the environment should be done by the Web application	Cannot send data properly because Web app never react it.

7.2 Sample of a Test Case for a Sensor

Working of sensors: It sends the data to Arduino and should sense the environment.

Description: Sense the data and fetch the environment condition by providing sensor to Arduino for a e.g. Sending data of temperature, Humidity and dew point by DHT11.

Precondition: Arduino was inactive.

Assumption: Sense the information and give it to Arduino by each pf the sensors.

Test Steps:

1. All the sensors be connected to Arduino board.
2. We also used to embed programmed to connect on board with the same pin.
3. Power supply should be provided to the Arduino board.
4. When you want to see the output of the programmed by using serial monitor.

Expected Result: each sensor should sense the information and get the earth condition give to Arduino.

Actual Result: Same as the Expected result.

Integration of sensors: It should send the data to thingspeak and sense the environment.

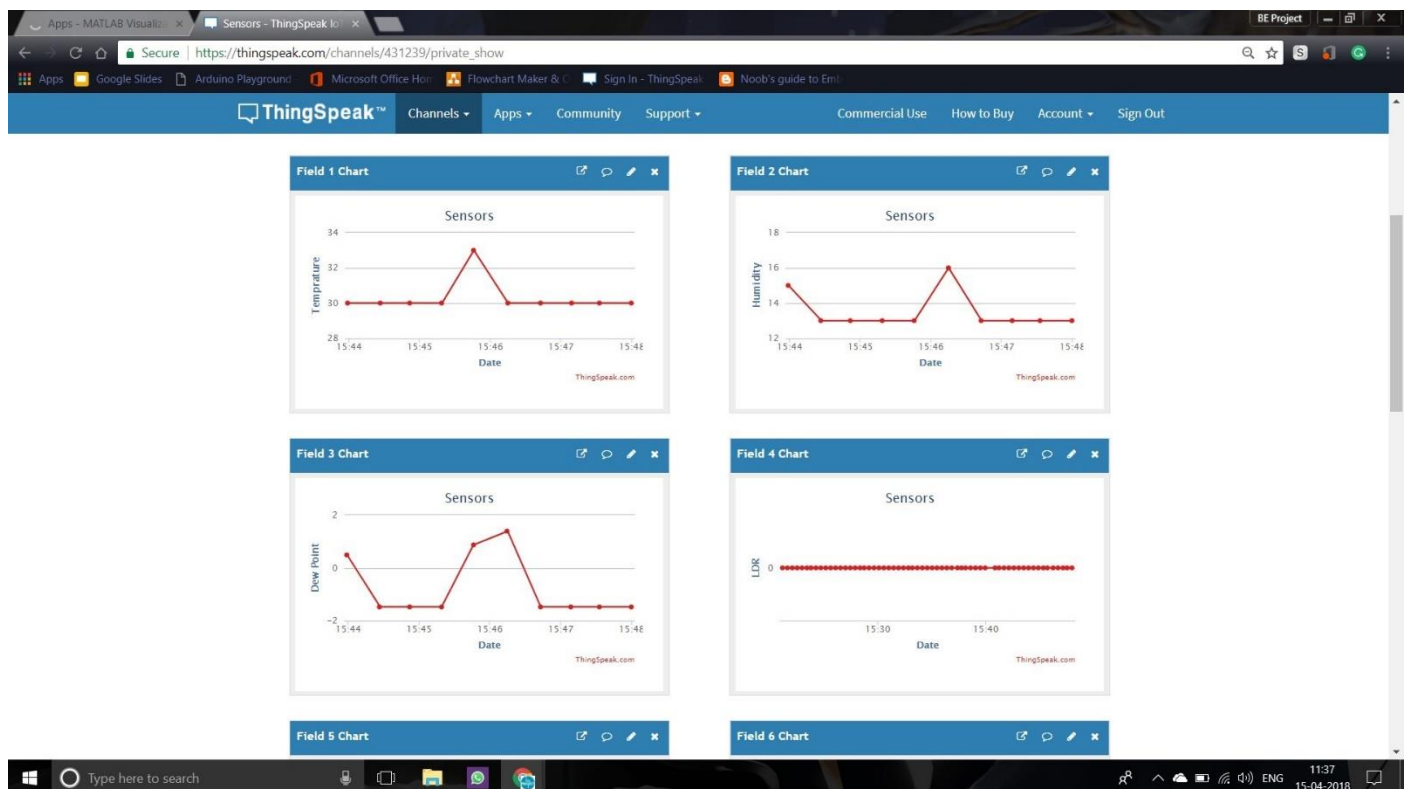
Description: Settlement of sensors – sense nature and information send to thingspeak making graphical portrayal and getting information from thingspeak. It uses ESP8622 Wi-Fi module sending the data via internet to the server. Arduino will be deployed ESP8266 Wi-Fi module sending information via web to the server. Data and create a graphical representation by Thingspeak.

Precondition: When the Wi-Fi should be on, can access ESP8266 the name and password of the hotspot must be having code.

Assumption: Using graphical format data may be represented by the server.

Test Steps:

1. Sensors should be connected to Arduino board.
2. The connection for TX and RX be removed from the Arduino board.
3. You should use the board to connect embed program with the same pin.
4. TX and RX pin be connected back to Arduino.
5. Providing power should be supplying to the Arduino board.
6. If you want to check the working, logging to the thinkspeak.



7.3 Software Quality Attributes

- **Availability:** Not to down the system. Availability of the specific data getting the user. The user uses the system anytime.
- **Correctness:** Searching correctness of data be shown to user.
- **Maintainability:** The system should be maintained by the administrator of the system.
- **Reliability:** For producing correct output, the system should be reliable. It is done by the user.
- **Extensibility:** System should be modified by changing some modules or adding some features to the existing system.

Chapter 8 – Conclusion & Future Scope

Need of agriculture the ways toward to make move relevant it. In new era, technology do great work its improvement. Its responsibility in management and professional agricultural sector. We face lots of problems in conventional farming. Create a relevant environment for the proper growth of plants with the help of insecticide and pesticide free crops. In future era, the smart customer connects directly with farmer. It is so important because the farmer knows how to improve the quality of crops, requirements of end users.

8.1 Future scope

- With the help of battery, Systems performance can be modified. Providing the power supply, battery can be rechargeable as need.
- Shade and fire detection condition can be including instead of measuring the conditions. Set up alarm can be reacting via voice command.
- Growth of crops data maintenance can be used using RFID tagging.

Chapter 9 – Work Load Matrix

Index Number	Arduino Programming	Server implementation	Web site designing and database	Wi-Fi connectivity	Hardware
10638293	✖				✖
10638234		✖			✖
10638231			✖		
10638293		✖	✖		
10642823	✖			✖	

Chapter 10 – Peer Evaluation

Evaluation criteria	10638231	10638234	10638292	10638293	10642823
Attends group meeting regularly and arrives on time.	4	4	4	4	4
Contributes meaningfully to group discussions.	4	4	4	4	4
Completes the tasks on time.	4	4	4	4	4
Prepares work in a quality manner.	4	4	4	4	4
Contributes significantly to the success of the project in a cooperative and supportive	4	4	4	4	4
TOTAL	20	20	20	20	20

References

There are no sources in the current document.